

CSC 830 - Advanced Artificial Intelligence

Lecture 05: Nets and Basic Search

Prof. Nwojo Agwu Nnanna
nagwu@nileuniversity.edu.ng

March 3, 2025

Nile University of Nigeria, Abuja

1. Nets and Basic (Uninformed) Search

Nets and Basic (Uninformed)Search

- Depth-first Search
- Breadth-first search

Constructing Goal-based Agents

- We have a **goal** to reach
 - Driving from point A to point B
 - Put 8 queens on a chess board such that no one attacks another
 - Prove that Achebe is an ancestor of Chinua
- We have information about where we are now at the **beginning**
- We have a set of **actions** we can take to move around (change from where we are)
- **Objective:** find a sequence of legal actions which will bring us from the start point to a goal

What is the goal to be achieved?

- Could describe a situation we want to achieve, a set of properties that we want to hold, etc.
- Requires defining a “**goal test**” so that we know what it means to have achieved/satisfied our goal.
- This is a hard part that is rarely tackled in AI, usually assuming that the system designer or user will specify the goal to be achieved.

What are the actions?

- Quantify all of the primitive actions or events that are sufficient to describe all necessary changes in solving a task/goal.
- No uncertainty associated with what an action does to the world. That is, given an action (aka operation or move) and a description of the current state of the world, the action completely specifies
 - **Precondition:** if that action CAN be applied to the current world (i.e., is it applicable and legal), and
 - **Effect:** what the exact state of the world will be after the action is performed in the current world (i.e., no need for "history" information to be able to compute what the new world looks like).

- Note also that actions can all be considered as **discrete events** that can be thought of as occurring at an **instant of time**.
 - That is, the world is in one situation, then an action occurs and the world is now in a new situation. For example, if "Ngozi is in class" and then performs the action "go home," then in the next situation she is "at home." There is no representation of a point in time where she is neither in class nor at home (i.e., in the state of "going home").
- The number of operators needed depends on the **representation** used in describing a state.

- Generally, a **representation** is a set of conventions about how to describe a class of things.
- **Description:** A description makes use of the conventions of a representation to describe some particular thing.
- Finding the appropriate representation is a major part of problem solving.

Example

A farmer is faced with the problem of moving himself and his possessions - a lion, a goat and some tubers of yam - across a river. He has a tiny boat that can take only two things (including the rower) at a time. Of course, only the farmer can row the boat. If the lion is left unattended with the goat, the lion will eat the goat. The goat would eat the yams if left unattended with the yams. How does the farmer transport himself and his possessions so that the lion does not eat the goat and the goat does not eat the yams?

The Representation Principle

- Once a problem is described using an appropriate representation, the problem is almost solved.
- A representation has four parts:
 - A **lexical** part: determines which symbols are allowed in the representation's vocabulary
 - A **structural** part: describes constraints on how the symbols can be arranged
 - A **procedural** part: specifies access procedures that enable you to create descriptions, to modify them, and to answer questions using them.
 - A **semantic** part: establishes a way of associating meaning with the descriptions.

- In the representation used to solve the farmer, lion, goat and yam problem:
 - Lexical part: determines that nodes and links are used.
 - Structural part: determines that links connect node pairs.
 - Semantic Part: establishes that nodes correspond to arrangements of the farmer, and his possessions and links correspond to river traversals.
 - Procedural part: is open.

- Is search the best way to solve the problem?
- Which search method solves the problem?
- Which search method is the most efficient for this problem?
- Applications
 - Computer networks
 - Robot motion
 - Network of cities

Net search is a tree search

- First, convert a network to a search tree. A search tree is a semantic tree in which each node denotes a path and branches connect paths to one-step path extensions.

Representing States

- At any moment, the relevant world is represented as a state
 - Initial (start) state: S
 - An action (or an operation) changes the current state to another state (if it is applied): state transition
 - An action can be taken (applicable) only if its precondition is met by the current state
 - For a given state, there might be more than one applicable actions
 - Goal state: a state satisfies the goal description or passes the goal test
 - Dead-end state: a non-goal state to which no action is applicable

- State space:
 - Includes the initial state S and all other states that are reachable from S by a sequence of actions
 - A state space can be organized as a graph:
 - nodes: states in the space
 - arcs: actions/operations
- The size of a problem is usually described in terms of the number of states (or the size of the state space) that are possible.
 - Tic-Tac-Toe has about 3^9 states.
 - Checkers has about 10^{40} states.
 - Chess has about 10^{120} states in a typical game.

Closed World Assumption

- We will generally use the Closed World Assumption.
- All necessary information about a problem domain is available in each percept so that each state is a complete description of the world.
- There is no incomplete information at any point in time.

Knowledge Representation Issues

- What's in a state?
 - Is the color of the boat relevant to solving the Missionaries and Cannibals problem? Is sunspot activity relevant to predicting the stock market? What to represent is a very hard problem that is usually left to the system designer to specify.
- What **level of abstraction** or detail to describe the world.
 - Too fine-grained and we'll "miss the forest for the trees." Too coarse-grained and we'll miss critical details for solving the problem.
- The number of states depends on the representation and level of abstraction chosen.
 - In the Remove-5-Sticks problem, if we represent the individual sticks, then there are 17-choose-5 possible ways of removing 5 sticks. On the other hand, if we represent the "squares" defined by 4 sticks, then there are 6 squares initially and we must remove 3 squares, so only 6-choose-3 ways of removing 3 squares.

Some Example Problems

- Toy problems and micro-worlds
 - 8-Puzzle
 - Missionaries and Cannibals
 - Cryptarithmic
 - Remove 5 Sticks
 - Traveling Salesman Problem (TSP)
- Real-world-problems

8-Puzzle

Given an initial configuration of 8 numbered tiles on a 3 x 3 board, move the tiles in such a way so as to produce a desired goal configuration of the tiles.

5	4	
6	1	8
7	3	2

Start State

1	2	3
8		4
7	6	5

Goal State

Figure 1: 8-Puzzle

8-Puzzle

- **State:** 3×3 array configuration of the tiles on the board.
- **Operators:** Move Blank square Left, Right, Up or Down.
 - This is a more efficient encoding of the operators than one in which each of four possible moves for each of the 8 distinct tiles is used.
- **Initial State:** A particular configuration of the board.
- **Goal:** A particular configuration of the board.
- The state space is partitioned into two subspaces
- NP-complete problem, requiring $O(2^k)$ steps where k is the length of the solution path.
- 15-puzzle problems (4×4 grid with 15 numbered tiles), and N -puzzles ($N = n^2 - 1$)

Place eight queens on a chessboard
such that no queen attacks any
other! Total # of states: 4.4×10^9
Total # of solutions: 12 (or 96)

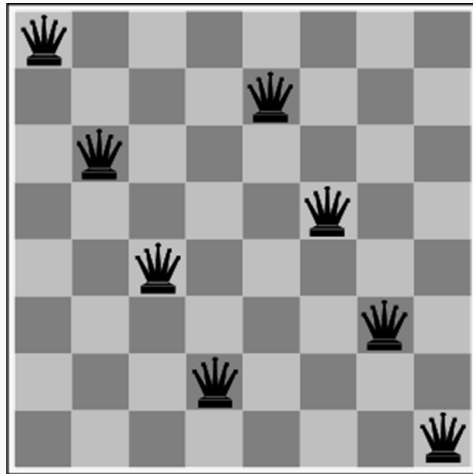
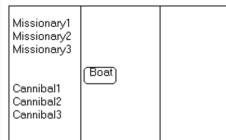


Figure 2: 8-Queens

Missionaries and Cannibals

There are 3 missionaries, 3 cannibals, and 1 boat that can carry up to two people on one side of a river.

- Goal: Move all the missionaries and cannibals across the river.
- Constraint: Missionaries can never be outnumbered by cannibals on either side of river, or else the missionaries are killed.
- State: configuration of missionaries and cannibals and boat on each side of river.
- Operators: Move boat containing some set of occupants across the river (in either



3 Missionaries and 3 Cannibals wish to cross the river. They have a boat that will carry two people. Everyone can navigate the boat. If at any time the Cannibals outnumber the missionaries on either bank of the river, they will eat the Missionaries. Find the smallest number of crossings that will allow everyone to cross the river safely. The problem can be solved in 11 moves. But people rarely get the optimal solution, because the MC problem contains a 'tricky' state at the end, where two people move back across the river.

Figure 3: Missionaries and Cannibals

Missionaries and Cannibals

		Near side	Far side
0	Initial setup	MMMCCC B	-

- Given a road map of n cities, find the shortest tour which visits every city on the map exactly once and then return to the original city (Hamiltonian circuit)
- (Geometric version):
 - a complete graph of n vertices.
 - $n!/2n$ legal tours
 - Find one legal tour that is shortest

Formalizing Search in a State Space

- A state space is a **graph**, (V, E) where V is a set of **nodes** and E is a set of **arcs**, where each arc is directed from a node to another node
- node: corresponds to a **state**
 - state description
 - plus optionally other information related to the parent of the node, operation to generate the node from that parent, and other bookkeeping data)
- arc: corresponds to an applicable action/operation.
 - the source and destination nodes are called as **parent (immediate predecessor)** and **child (immediate successor)** nodes with respect to each other
 - ancestors(predecessors) and descendents (successors)
 - each arc has a fixed, non-negative cost associated with it, corresponding to the **cost** of the action

- node generation: making explicit a node by applying an action to another node which has been made explicit
- node expansion: generate all children of an explicit node by applying all applicable operations to that node
- One or more nodes are designated as start nodes
- A goal test predicate is applied to a node to determine if its associated state is a goal state
- A solution is a sequence of operations that is associated with a path in a state space from a start node to a goal node
- The cost of a solution is the sum of the arc costs on the solution path

- State-space search is the process of searching through a state space for a solution by making explicit a sufficient portion of an implicit state-space graph to include a goal node.
 - Hence, initially $V=\{S\}$, where S is the start node; when S is expanded, its successors are generated and those nodes are added to V and the associated arcs are added to E . This process continues until a goal node is generated (included in V) and identified (by goal test)
- During search, a node can be in one of the three categories:
 - Not generated yet (has not been made explicit yet)
 - OPEN: generated but not expanded
 - CLOSED: expanded
 - Search strategies differ mainly on how to select an OPEN node for expansion at each step of search

A General State Space Search Algorithm

- Node n
 - state description
 - parent (may use a backpointer) (if needed)
 - Operator used to generate n (optional)
 - Depth of n (optional)
 - Path cost from S to n (if available)
- OPEN list
 - initialization: $\{S\}$
 - node insertion/removal depends on specific search strategy
- CLOSED list
 - initialization: $\{ \}$
 - organized by backpointers

A General State Space Search Algorithm

```
open := {S}; closed :={ }  
repeat  
  n := select(open); /* select one node from open for expansion */  
  if n is a goal  
    then exit with success; /* delayed goal testing */  
    expand(n)  
    /* generate all children of n  
    put these newly generated nodes in open (check duplicates)  
    put n in closed (check duplicates) */  
until open = { };  
exit with failure
```

Some Issues

- Search process constructs a search tree, where
 - root is the initial state S , and
 - leaf nodes are nodes
 - not yet been expanded (i.e., they are in OPEN list) or
 - having no successors (i.e., they're "deadends")
- Search tree may be infinite because of loops even if state space is small
- Search strategies mainly differ on select (open)
- Each node represents a partial solution path (and cost of the partial solution path) from the start node to the given node.
 - in general, from this node there are many possible paths (and therefore solutions) that have this partial path as a prefix.

Evaluating Search Strategies

- **Completeness**

- Guarantees finding a solution whenever one exists

- **Time Complexity**

- How long (worst or average case) does it take to find a solution?
Usually measured in terms of the **number of nodes expanded**

- **Space Complexity**

- How much space is used by the algorithm? Usually measured in terms of the **maximum size that the “OPEN” list** becomes during the search

- **Optimality/Admissibility**

- If a solution is found, is it guaranteed to be an optimal one? For example, is it the one with minimum cost?

Uninformed vs Informed Search Strategies

- Uninformed Search Strategies
 - Breadth-First search
 - Depth-First search
 - Uniform-Cost search
 - Depth-First Iterative Deepening search
- Informed Search Strategies
 - Hill climbing
 - Best-first search
 - Branch and bound
 - Branch and bound with underestimates
 - Branch and bound with dynamic programming
 - A*

Breadth-First Search (BFS)

- Algorithm outline:
 - Always select from the OPEN the node with the smallest depth for expansion, and put all newly generated nodes into OPEN
 - OPEN is organized as FIFO (first-in, first-out) list, i.e., a queue.
 - Terminate if a node selected for expansion is a goal
- Properties
 - Complete
 - Optimal (i.e., admissible) if all operators have the same cost. Otherwise, not optimal but finds solution with shortest path length (shallowest solution).
 - Exponential time and space complexity, $O(b^d)$ nodes will be generated, where d is the depth of the solution and b is the branching factor (i.e., number of children) at each node

- A complete search tree of depth d where each non-leaf node has b children, has a total of $1 + b + b^2 + \dots + b^d = (b^{(d+1)} - 1)/(b - 1)$ nodes
- Time complexity (# of nodes generated): $O(b^d)$
- Space complexity (maximum length of OPEN): $O(b^d)$
- For a complete search tree of depth 12, where every node at depths $0, \dots, 11$ has 10 children and every node at depth 12 has 0 children, there are $1 + 10 + 100 + 1000 + \dots + 10^{12} = (10^{13} - 1)/9 = O(10^{12})$ nodes in the complete search tree.
- BFS is suitable for problems with shallow solutions

Breadth-First Search: The Algorithm

To conduct a breadth-first search,

1. Form a one-element queue consisting of a zero-length path that contains only the root node
2. Until the first path in the queue terminates at the goal node or the queue is empty
 - I. Remove the first path from the queue; create new paths by extending the first path to all the neighbors of the terminal node.
 - II. Reject all new paths with loops.
 - III. Add the new paths, if any, to the **back** of the queue.
3. If the goal node is found, announce success; otherwise, announce failure.

LISP Implementation of the Breadth-First Search Algorithm

```
(defun breadth-first-search (start finish &optional
                             (path-list (list (list start))))
  (format t "~%List of paths is ~a" path-list)
  (cond ((endp path-list) nil) ; empty path-list
        ((equal finish (first (first path-list))) ;goal found
         (reverse (first path-list)))
        (t (breadth-first-search start finish ;keep searching
                                   (append (rest path-list); add new paths to REAR of list
                                   (extend (first path-list)))))))
```

Depth-First Search (DFS)

- Algorithm outline:
 - Always select from the OPEN the node with the greatest depth for expansion, and put all newly generated nodes into OPEN
 - OPEN is organized as LIFO (last-in, first-out) list.
 - Terminate if a node selected for expansion is a goal
- May not terminate without a "depth bound," i.e., cutting off search below a fixed depth D (How to determine the depth bound?)
- Not complete (with or without cycle detection, and with or without a cutoff depth)
- Exponential time, $O(b^d)$, but only linear space, $O(bd)$, required
- Can find deep solutions quickly if lucky
- When search hits a deadend, can only back up one level at a time even if the "problem" occurs because of a bad operator choice near the top of the tree. Hence, only does "chronological backtracking"

Depth-First Search: The Algorithm

To conduct a breadth-first search,

1. Form a one-element queue consisting of a zero-length path that contains only the root node
2. Until the first path in the queue terminates at the goal node or the queue is empty
 - I. Remove the first path from the queue; create new paths by extending the first path to all the neighbors of the terminal node.
 - II. Reject all new paths with loops.
 - III. Add the new paths, if any, to the **front** of the queue.
3. If the goal node is found, announce success; otherwise, announce failure.

LISP Implementation of the Depth-First Search Algorithm

```
(defun breadth-first-search (start finish &optional
                             (path-list (list (list start))))
  (format t "~%List of paths is ~a" path-list)
  (cond ((endp path-list) nil) ; empty path-list
        ((equal finish (first (first path-list))) ;goal found
         (reverse (first path-list)))
        (t (depth-first-search start finish ;keep searching
                                (append (extend (first path-list)); add new paths to REAR of list
                                (rest path-list))))))
```


The function: extend

```
(defun extend (path)
  (print (reverse path))
  (mapcar #'(lambda (new-node)
              (cons new-node path))
          (remove-if #'(lambda (neighbor)
                        (member neighbor path))
                    (get (first path) 'neighbors)))))
```

Depth-First Iterative Deepening (DFID)

- BF and DF both have exponential time complexity $O(b^d)$
 - BF is complete but has exponential space complexity
 - DF has linear space complexity but is incomplete
- Space is often a harder resource constraint than time
- Can we have an algorithm that
 - Is complete
 - Has linear space complexity, and
 - Has time complexity of $O(b^d)$
- DFID by Korf in 1985 (17 years after A*)

First do DFS to depth 0 (i.e., treat start node as having no successors), then, if no solution found, do DFS to depth 1, etc.

until solution found do

DFS with depth bound c

$c = c + 1$

Depth-First Iterative Deepening (DFID)

- Complete (iteratively generate all nodes up to depth d)
- Optimal/Admissible if all operators have the same cost. Otherwise, not optimal but does guarantee finding solution of shortest length (like BF).
- Linear space complexity: $O(bd)$, (like DF)
- Time complexity is a little worse than BFS or DFS because nodes near the top of the search tree are generated multiple times, but because almost all of the nodes are near the bottom of a tree, the worst case time complexity is still exponential, $O(b^d)$

Depth-First Iterative Deepening (DFID)

- If branching factor is b and solution is at depth d , then nodes at depth d are generated once, nodes at depth $d-1$ are generated twice, etc., and node at depth 1 is generated d times. Hence

$$\begin{aligned} \text{total}(d) &= b^d + 2b^{d-1} + \dots + db \\ &\leq b^d / (1 - 1/b)^2 = O(b^d). \end{aligned}$$

- If $b=4$, then worst case is $1.78 * 4^d$, i.e., 78% more nodes searched than exist at depth d (in the worst case).

$$\begin{aligned}
 total(d) &= 1.b^d + 2.b^{d-1} + \dots + (d-1).b^2 + d.b \\
 &= b^d(1 + 2b^{-1} + \dots + (d-1)b^{2-d} + d.b^{1-d})
 \end{aligned}$$

Let $x = b^{-1}$. Then

$$\begin{aligned}
 total(d) &= b^d(1 + 2x^1 + \dots + (d-1)x^{d-2} + dx^{d-1}) \\
 &= b^d \frac{d}{dx} (x + x^2 + \dots + x^{d-1} + x^d) \\
 &= b^d \frac{d}{dx} \frac{x - x^{d+1}}{1 - x} \\
 &\leq b^d \frac{d}{dx} \frac{x}{1 - x}, \quad / * x^{d+1} \ll 1 \text{ when } d \text{ is large} * / \\
 &= b^d \frac{(1-x) - x(-1)}{(1-x)^2}.
 \end{aligned}$$

Threorefore

$$total(d) \leq \frac{b^d}{(1-x)^2} = \frac{b^d}{(1-b^{-1})^2}.$$

When to use what

- **Depth-First Search:**
 - Many solutions exist
 - Know (or have a good estimate of) the depth of solution
- **Breadth-First Search:**
 - Some solutions are known to be shallow
- **Iterative-Deepening Search:**
 - Space is limited and the shortest solution path is required

References

- [1] P. H. Winston, Artificial Intelligence, Addison Wesley.
- [2] George F Luger, Artificial Intelligence, Addison Wesley